

```

# bot.py
import telebot
from telebot import types
import random
import sqlite3
import os
import time

# TOKEN (أمن) سيأخذ قيمة من متغير بيئة في Render
TOKEN = os.environ.get("TELEGRAM_TOKEN")
if not TOKEN:
    raise RuntimeError("Please set TELEGRAM_TOKEN environment variable")

bot = telebot.TeleBot(TOKEN)

DB_PATH = "halha_bot.db"

# --- Helpers: قاعدة بيانات بسيطة SQLite ---
def init_db():
    conn = sqlite3.connect(DB_PATH)
    cur = conn.cursor()
    cur.execute("""CREATE TABLE IF NOT EXISTS users (
        user_id INTEGER PRIMARY KEY,
        username TEXT,
        display_name TEXT,
        total_points INTEGER DEFAULT 0,
        last_played INTEGER DEFAULT 0
    )""")
    conn.commit()
    conn.close()

def add_or_update_user(user_id, username, display_name, points):
    conn = sqlite3.connect(DB_PATH)
    cur = conn.cursor()
    cur.execute("INSERT OR IGNORE INTO users (user_id, username, display_name,
total_points, last_played) VALUES (?, ?, ?, ?, ?)",
        (user_id, username, display_name, 0, int(time.time())))
    cur.execute("UPDATE users SET total_points = total_points + ?, last_played = ? WHERE
user_id = ?",
        (points, int(time.time()), user_id))
    conn.commit()
    conn.close()

def top_leaderboard(limit=10):

```

```

conn = sqlite3.connect(DB_PATH)
cur = conn.cursor()
cur.execute("SELECT display_name, total_points FROM users ORDER BY total_points
DESC LIMIT ?", (limit,))
rows = cur.fetchall()
conn.close()
return rows

```

--- Bank of questions (يمكن تعديل / توسيع لاحقاً) ---

```

QUESTIONS = [
    {"q": "كم تبقى معك؟", "choices": ["A) واحدة", "B) اثنتان", "C) صفر"], "answer": "A"},
    {"q": "كم عدد درجات الزوايا في مثلث؟", "choices": ["A) 90", "B) 180", "C) 360"], "answer": "B"},
    {"q": "ما اسم عاصمة العراق؟", "choices": ["A) البصرة", "B) الموصل", "C) بغداد"], "answer": "C"},
    {"q": "كم ضلع للمستطيل؟", "choices": ["A) 3", "B) 4", "C) 5"], "answer": "B"},
    {"q": "ترجمة كلمة 'كتاب' إلى الإنجليزية؟", "choices": ["A) Book", "B) Table", "C) Pen"], "answer": "A"},
    {"q": "أي مما يلي يساعد على تنظيم الدراسة؟", "choices": ["A) التسويف", "B) التخطيط", "C) تجاهل المراجعة"], "answer": "B"},
]

```

--- لجولات الأسئلة (in-memory) جلسة مستخدم مؤقتة ---

```
sessions = {}
```

```
def new_session(chat_id):
```

```
    # يختار 3 أسئلة عشوائية
```

```
    qs = random.sample(QUESTIONS, k=3 if len(QUESTIONS)>=3 else len(QUESTIONS))
```

```
    sessions[chat_id] = {"questions": qs, "index":0, "score":0}
```

```
    return sessions[chat_id]
```

--- رسائل البداية والأزرار ---

```
START_TEXT = "" أهلاً بالمُحلّل الذكي 🙌
```

```
بوت التحدي لـ #مرماز_تحلها — HalhaAI أنا
```

♦ اضغط ابدأ وخوض 3 أسئلة سريعة

♦ اربح نقاطاً وارفع ترتيبك في اللوحة

```
"""
```

```
@bot.message_handler(commands=['start'])
```

```
def handle_start(msg):
```

```
    chat_id = msg.chat.id
```

```
    markup = types.ReplyKeyboardMarkup(row_width=2, resize_keyboard=True)
```

```
    markup.add("ابدأ التحدي 🎮", "الترتيب")
```

```
    markup.add("شارك البوت 📧", "عن المسابقة")
```

```
    bot.send_message(chat_id, START_TEXT, reply_markup=markup)
```

```

@bot.message_handler(func=lambda m: m.text == "ابدأ التحدي 🎮")
def handle_start_challenge(msg):
    chat_id = msg.chat.id
    s = new_session(chat_id)
    send_question(chat_id)

def send_question(chat_id):
    s = sessions.get(chat_id)
    if not s:
        bot.send_message(chat_id, "اضغط 'ابدأ التحدي' لبدء الجولة.")
        return
    idx = s["index"]
    if idx >= len(s["questions"]):
        finish_session(chat_id)
        return
    q = s["questions"][idx]
    markup = types.ReplyKeyboardMarkup(row_width=1, resize_keyboard=True,
one_time_keyboard=True)
    for choice in q["choices"]:
        markup.add(choice)
    bot.send_message(chat_id, f" ? سؤال {idx+1}: \n{q['q']}", reply_markup=markup)

```

```

@bot.message_handler(func=lambda m: True)
def handle_answer(msg):
    text = msg.text.strip()
    chat_id = msg.chat.id
    # الأوامر الأساسية
    if text == "الترتيب 📊":
        show_leaderboard(chat_id)
        return
    if text == "شارك البوت ✉️":
        bot.send_message(chat_id, f"شارك هذا الرابط ودع أصدقائك يجربون:
https://t.me/{bot.get_me().username}")
        return
    if text == "عن المسابقة ⓘ":
        bot.send_message(chat_id, "بوت تحدي صغير تابع لحملة #مرماز_تحلها — تجربة تفاعلية بسيطة")
        return

    s = sessions.get(chat_id)
    if not s:
        bot.send_message(chat_id, "اضغط 'ابدأ التحدي' لبدء الجولة.")
        return

    idx = s["index"]

```

```

if idx >= len(s["questions"]):
    bot.send_message(chat_id, "الجولة انتهت، اضغط 'ابدأ التحدي' لجولة جديدة.")
    return

correct_option = s["questions"][idx]["answer"]
# أو الاختيار كامل (A/B/C) نأخذ الحرف الأول من الإجابة المرسله
user_choice = text.strip()
# normalize: "A" واحدة "نحول إلى (A) إذا أرسل المستخدم النص
if len(user_choice) > 0 and user_choice[0].upper() in ["A", "B", "C"]:
    user_choice_char = user_choice[0].upper()
else:
    # نقارن بالخيارات لمعرفة أي واحدة اختار
    # نبحث أي خيار يبدأ بنفس النص
    user_choice_char = None
    for opt in s["questions"][idx]["choices"]:
        if user_choice in opt or user_choice.lower() in opt.lower():
            user_choice_char = opt[0].upper()
            break

if user_choice_char == correct_option:
    bot.send_message(chat_id, "صح! +1 نقطة ✅")
    s["score"] += 1
else:
    correct_text = next((c for c in s["questions"][idx]["choices"] if c[0].upper() == correct_option),
    "")
    bot.send_message(chat_id, f"خطأ! الجواب الصحيح ❌ {correct_text}")

s["index"] += 1
# إرسال السؤال التالي أو انتهاء
if s["index"] < len(s["questions"]):
    send_question(chat_id)
else:
    finish_session(chat_id)

def finish_session(chat_id):
    s = sessions.get(chat_id)
    points = s["score"] if s else 0
    # حفظ في DB
    try:
        user = bot.get_chat(chat_id)
        username = user.username or ""
        display = user.first_name or username or str(chat_id)
        add_or_update_user(chat_id, username, display, points)
    except Exception as e:

```

```

print("DB save error:", e)

# رسالة النتيجة مع أزرار
markup = types.InlineKeyboardMarkup()
markup.add(types.InlineKeyboardButton("شارك نتيجتي 🇸🇦", switch_inline_query=f"أنا حلّيتها"
{points}/3 #رماز تحلها"))
markup.add(types.InlineKeyboardButton("لوحة المتصدرين 🏆",
callback_data="show_leaderboard"))
bot.send_message(chat_id, f"🎉 انتهت الجولة! نقاطك {points}/3", reply_markup=markup)
# ننظف الجلسة
if chat_id in sessions:
    del sessions[chat_id]

@bot.callback_query_handler(func=lambda call: True)
def callback_query(call):
    if call.data == "show_leaderboard":
        rows = top_leaderboard(10)
        text = "🏆 لوحة المتصدرين\n"
        for i, r in enumerate(rows, start=1):
            text += f"{i}. {r[0]} — {r[1]}\n"
        bot.edit_message_text(text, call.message.chat.id, call.message.message_id)

# init
if __name__ == "__main__":
    init_db()
    print("Bot started, polling...")
    bot.infinity_polling(timeout=60, long_polling_timeout=5)

```